

Classification of handwritten digits

We will use the data from the US Post Office Zip Code Data https://web.stanford.edu/~hastie/StatLearnSparsity_files/DATA/zipcode.html consisting of two data bases: the training set, that contains 7291 handwritten numbers, and the test set, containing 2007 handwritten numbers. They are all taken from the handwritten zip code of letters from the US.

The dataset is provided in the file `zipdata.mat`, which includes the following matrices:

- `azip.mat`: A tensor of size $16 \times 16 \times 7291$. Each slice `azip(:,:,i)` represents the i -th handwritten digit in the training set.
- `testzip.mat`: A tensor of size $16 \times 16 \times 2007$. Each slice `testzip(:,:,i)` represents the i -th handwritten digit in the test set.
- `dzip.mat`: A 1×7291 row vector containing the corresponding digit label for each matrix in the training set.
- `dtest.mat`: A 1×2007 row vector containing the corresponding digit label for each matrix in the test set.

Each 16×16 matrix in `azip.mat` and `testzip.mat` contains grayscale pixel intensity values ranging from -1 to 1 , where the values represent the intensity of the handwritten strokes.

To visualize a digit, the following MATLAB function can be used:

```
function []=ima(A)
    a1 = squeeze(A);
    a1 = (a1 - min(min(a1)) * ones(size(a1)));
    a1 = (20 / max(max(a1))) * a1;
    colormap(gray);
    image(a1);
    axis off;
    axis equal;
end
```

By calling `ima(azip(:,:,500))`, for example, one can visualize the 500th digit from the training set.

1. Use the information in the matrix `dzip.mat` to classify all digits in the training matrix (in 10 sets containing, respectively, the digits from 0 to 9).

We have the true digit values for each matrix, it is simply a matter of assigning each one to the corresponding subset.

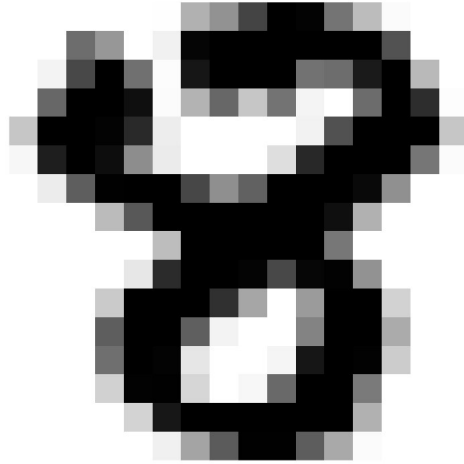
```
1 % 1. Classify training set into 10 sets
2 subsets = cell(10, 1);
3 data_flattened = reshape(azip, 256, []);
4 % Iterate over each digit class (0 to 9)
```

```
5 for digit=0:9
6     indices = find(dzip == digit);
7     subsets{digit + 1} = data_flattened(:, indices);
8 end
```

To check the code used to classify is correct, we can plot (using the provided `ima` function) a random digit of choice using this code:

```
1 %% CHECK CLASSIFICATION IS CORRECT BY PLOTTING RANDOM NUMBER
  IN SUBSETS
2
3 % Extract the subset for digit
4 digit = 8; % Choose the digit (8 for example)
5 subset = subsets{digit + 1};
6
7 % Select a specific sample from the subset from 1 to n_i
8 sample_index = 64;
9 sample = subset(:, sample_index); % Extract the sample (column
  vector)
10
11 % Reshape the sample to its original 16x16 form
12 sample_image = reshape(sample, [16, 16]);
13
14 % Display the sample using the ima function
15 figure;
16 ima(sample_image);
```

We arbitrarily chose the 1st digit of the subset of digit 8, giving the following image:



We repeated this procedure several times and varying digits, and it seems that the classification is correct.

2. Write a MATLAB function for the “SVD basis classification algorithm”, taking the left singular vectors corresponding to the 10 largest singular values of the matrix associated to each digit set from the previous item.

```
1 % 2. SVD basis classification algorithm (10 largest singular
  values)
2 function [predictions, rel_residuals] = svd_algorithm(subsets,
  testzip, N, k)
3     % N : number of digits of test data to be predicted
4     % k : number of singular values to retain
5     svd_results = cell(10, 1);
6     rel_residuals = zeros(k, N); % Store residuals
7     for digit=0:9
8         matrix = subsets{digit+1};
9         [U, ~, ~] = svd(matrix);
10        U_k = U(:, 1:k);
11        svd_results{digit + 1} = U_k;
12    end
13
14    % Extract the first N digits (for example)
15    first_N_digits = testzip(:, :, 1:N); % (16x16xN)
16
17    % Reshape into a 256xN matrix
18    test_set = reshape(first_N_digits, [256, N]); % Reshape to
    256xN
19    predictions = zeros(1, N); % Store predictions
```

```

20
21     for i=1:20
22         distances = zeros(1, k);
23         for digit=0:9
24             U = svd_results{digit+1};
25             distances(digit+1) = norm(test_set(:,i)-U*(U'*
26                 test_set(:,i)), 2);
27             rel_residuals(digit+1,i)=distances(digit+1) / norm(
28                 test_set(:, i), 2);
29         end
30         % Find the digit with the smallest distance
31         [~, predicted_digit] = min(distances); % Get the index
32         % of the smallest distance
33         predictions(1, i) = predicted_digit - 1; % Subtract 1
34         % to match digit
35     end
36 end

```

3. Run the code of the previous item on 20 vectors from the test set, and comment on whether the image corresponding to each of these vectors is in accordance with the output of the algorithm.

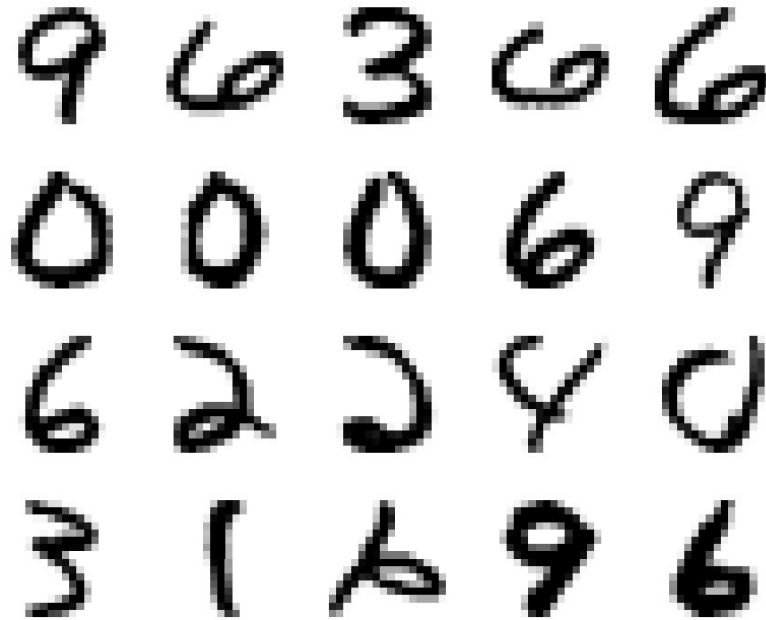
We arbitrarily choose the first 20 digits from the test set and plot them using the following code:

```

1  %%% PLOT FIRST 20 DIGITS OF TEST SET
2  % Create a figure
3  figure(4);
4  colormap(gray); % Set grayscale colormap
5
6  % Loop through the first 20 digits and plot them
7  for i = 1:20
8      I = first_20_digits(:, :, i); % Extract the i-th digit (16
9          x16)
10      im=mat2gray(1-I);
11      subplot(4, 5, i); % Arrange plots in a 4x5 grid
12      imshow(im);
13      axis off;
14  end

```

We obtain these digits:



By executing the function in Part 2:

```
1 [predictions, ~] = svd_algorithm(subsets, testzip, 20, 10);
```

we obtain the following predictions for these 20 digits:

```
1 predictions =  
2  
3 9 6 3 6 6 0 0 0 6 9 6 2 3 8 0 3 1 2 9 6
```

By inspection, 17 out of 20 digits are correctly classified, giving an accuracy of 85%. Alternatively, we use MATLAB commands to compare the predictions from the true digit values in `dtest`:

```
1 % Calculate the number of correct predictions  
2 num_correct = sum(predictions == dtest(1:20));  
3 % Compute accuracy as a percentage  
4 accuracy = (num_correct / length(predictions)) * 100;  
5 % Display the accuracy  
6 fprintf('Accuracy: %.2f%%\n', accuracy);
```

This gives:

```
1 Accuracy: 85.00%
```

4. Measure the relative residual of each of the 20 vectors in the previous item and plot them. Is there a noticeable difference between the smallest one and the other ones in each case?

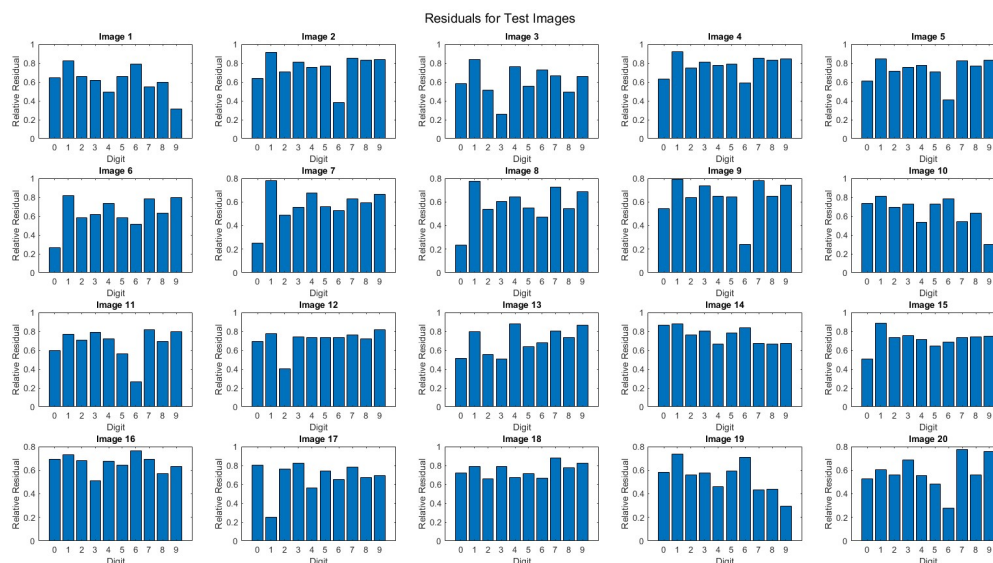
We use the previously implemented `svd_algorithm` function to calculate the residuals associated with the 20 vectors:

```

1 % Compute residuals
2 [~, rel_residuals] = svd_algorithm(subsets, testzip, 20, 10);
3
4 [num_digits, num_vectors] = size(rel_residuals);
5 figure;
6 for i = 1:num_vectors
7     subplot(4, 5, i); % Create a 4x5 grid of subplots
8     bar(0:num_digits-1, rel_residuals(:, i));
9     xlabel('Digit');
10    ylabel('Relative Residual');
11    title(sprintf('Vector %d', i));
12 end
13 sgtitle('Residuals for Test Vectors'); % Add a global title

```

This gives:



In each test image, the digit associated with the smallest residual corresponds to the one assigned. Residuals associated with images that are more difficult to classify are more similar to each other. We note that in some instances, such that in Image 3 (which is very obviously a 3) the residual corresponding to the assigned digit is considerably smaller than the rest.